

Bài 17: Xử lý File và Serialization trong Java

Java cung cấp các class để làm việc với file và lưu trữ đối tượng lâu dài thông qua serialization.

1. Các class xử lý file

File

- Đại diện cho đường dẫn file hoặc thư mục, không trực tiếp đọc/ghi nội dung.
- Dùng để kiểm tra, tạo, xóa hoặc lấy thông tin file.

```
File file = new File("data.txt");
file.createNewFile();
System.out.println(file.exists());
System.out.println(file.getName());
file.delete();
```

FileReader

- Đọc dữ liệu ký tự từ file.
- Phù hợp với file văn bản nhỏ.

```
try (FileReader reader = new FileReader("data.txt")) {
    int ch;
    while ((ch = reader.read()) != -1) {
        System.out.print((char) ch);
    }
}
```

FileWriter

- Ghi dữ liệu ký tự vào file.
- Truyền `true` ở tham số thứ hai để ghi nối tiếp (append).

```
try (FileWriter writer = new FileWriter("data.txt", true)) {
    writer.write("Hello Java\n");
}
```

BufferedReader

- Đọc văn bản hiệu quả theo từng dòng.
- Dùng bộ đệm nên nhanh hơn khi đọc nhiều dữ liệu.

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
}
```

BufferedWriter

- Ghi văn bản hiệu quả theo từng dòng.
- Dùng bộ đệm nên nhanh hơn khi ghi nhiều dữ liệu.

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter("data.txt"))) {
    bw.write("Hello Java");
    bw.newLine();
}
```

2. Serialization

Serialization là quá trình chuyển trạng thái của đối tượng thành một luồng byte để lưu vào file hoặc gửi qua mạng.

Các bước

1. Đối tượng phải implement interface `Serializable`.
2. Dùng `ObjectOutputStream` để ghi đối tượng vào file.

```
import java.io.Serializable;

class Student implements Serializable {
    private static final long serialVersionUID = 1L;

    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

```
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;

Student student = new Student("Minh", 25);
```

```
try (FileOutputStream fos = new FileOutputStream("student.ser");
     ObjectOutputStream oos = new ObjectOutputStream(fos)) {
    oos.writeObject(student);
}
```

Student (name, age) — Serialization → File (student.ser)

3. Deserialization

Deserialization là quá trình ngược lại của serialization: chuyển luồng byte từ file thành đối tượng.

Các bước

1. Dùng `ObjectInputStream` để đọc đối tượng từ file.
2. Ép kiểu kết quả về đúng kiểu đối tượng đã lưu.

```
import java.io.FileInputStream;
import java.io.ObjectInputStream;

try (FileInputStream fis = new FileInputStream("student.ser");
     ObjectInputStream ois = new ObjectInputStream(fis)) {
    Student s2 = (Student) ois.readObject();
    System.out.println(s2.name + " - " + s2.age);
}
```

File (student.ser) — Deserialization → Student (name, age)

4. Điểm chính

- Class cần được lưu trữ phải implement `Serializable`.
- Dùng class có đệm (`BufferedReader`, `BufferedWriter`) để đọc/ghi hiệu quả hơn.
- Serialization giúp lưu lại trạng thái đối tượng.
- Deserialization tạo lại đối tượng từ dữ liệu đã lưu.
- Field `transient` và `static` không được serialization.
- `serialVersionUID` giúp kiểm soát tính tương thích khi class thay đổi.

5. Thực hành tốt

- Luôn đóng stream sau khi dùng; ưu tiên `try-with-resources` để tự động đóng.
- Không serialize khóa bí mật, mật khẩu hoặc dữ liệu nhạy cảm.
- Dùng stream có đệm khi làm việc với file lớn.
- Kiểm tra đường dẫn file và xử lý ngoại lệ cẩn thận.

6. Các ngoại lệ thường gặp

Ngoại lệ	Nguyên nhân
<code>FileNotFoundException</code>	Không tìm thấy file hoặc không thể mở file
<code>IOException</code>	Lỗi nhập/xuất chung
<code>ClassNotFoundException</code>	Không tìm thấy class khi deserialization
<code>InvalidClassException</code>	Class không tương thích với dữ liệu đã serialize

7. Ghi nhớ

- Serialization: đối tượng → luồng byte → lưu.
- Deserialization: luồng byte → đối tượng → nạp.
- Cần phân biệt `Serializable` và `Externalizable`: `Externalizable` cho phép tự kiểm soát cách đọc/ghi dữ liệu nhưng phức tạp hơn.